Roadmap Backend — Bibliothèque Digitale

Projet PFA — Conception et réalisation d'une bibliothèque digitale des références et ressources numériques
Stack : Laravel 12 · React JS · MySQL 8.4 · Microservice IA (Python/FastAPI + Gemini + Chroma)

Phase 1 — Fondations (base de données)

1. Lancer les migrations et vérifier que toutes les tables sont créées

```
php artisan migrate
```

2. Créer les modèles Eloquent

```
php artisan make:model Category
php artisan make:model Resource
php artisan make:model ContributionRequest
php artisan make:model Review
php artisan make:model Favorite
php artisan make:model Consultation
php artisan make:model DocumentChunk
php artisan make:model Conversation
php artisan make:model Message
```

3. Définir les relations Eloquent dans chaque modèle (belongsTo, hasMany) — la traduction directe du diagramme de classes

4. Créer des Seeders pour avoir des données de test (catégories de base, utilisateurs avec des rôles différents)

```
php artisan make:seeder CategorySeeder
php artisan make:seeder UserSeeder
```

Test de fin de phase : via `php artisan tinker`, créer une ressource et vérifier que `$resource->category->name` fonctionne.

Phase 2 — Authentification et rôles

5. Installer et configurer Sanctum

```
composer require laravel/sanctum
php artisan install:api
```

6. Créer AuthController : register, login, logout, me

```
php artisan make:controller Api/AuthController
```

7. Installer Spatie Permission et créer les rôles (user, moderator, admin) dans un seeder

```
composer require spatie/laravel-permission
```

8. Créer les middlewares de protection des routes par rôle

Test de fin de phase : avec Postman, s'inscrire > se connecter > récupérer le token > appeler une route protégée. Vérifier qu'un utilisateur normal ne peut pas accéder à une route admin.

Phase 3 — CRUD de base

9. CategoryController : lister, créer, modifier, supprimer les catégories (protégé pour modérateur/admin)

10. ResourceController : lister, afficher, créer (avec upload), modifier, supprimer

11. Form Requests pour la validation propre

```
php artisan make:request StoreResourceRequest
```

12. API Resources pour formater les réponses JSON

```
php artisan make:resource ResourceResource
php artisan make:resource CategoryResource
```

13. Configurer CORS (config/cors.php) pour autoriser React

Test de fin de phase : créer / lister / modifier des catégories et des ressources via Postman.

Phase 4 — Upload de fichiers

14. Configurer PHP (php.ini) pour les uploads jusqu'à 100 Mo

```
upload_max_filesize = 100M
post_max_size = 110M
max_execution_time = 300
memory_limit = 256M
```

15. Rendre les fichiers accessibles publiquement

```
php artisan storage:link
```

16. Implémenter l'upload dans ResourceController@store : validation du type, stockage, calcul du file_hash

17. Détection de doublons : vérifier le hash avant insertion, retourner une erreur 409 si doublon

18. Route de téléchargement sécurisée

Test de fin de phase : uploader un PDF via Postman, vérifier qu'il arrive dans `storage/app/public/resources/`, et qu'un second upload du même fichier est rejeté.

Phase 5 — Workflow de modération

19. ContributionRequestController : lister les demandes en attente, valider, rejeter

20. Logique métier : à la validation, passer resource.status à 'validated' + enregistrer moderator_id et processed_at

21. Créer une ResourcePolicy pour les autorisations fines (qui peut modifier quoi)

```
php artisan make:policy ResourcePolicy --model=Resource
```

22. Route « mes contributions » pour que l'utilisateur suive le statut de ses dépôts

Test de fin de phase : déposer une ressource en tant qu'utilisateur, la valider en tant que modérateur, vérifier que le statut change et qu'elle apparaît dans le catalogue.

Phase 6 — Fonctionnalités utilisateur

23. FavoriteController : ajouter / retirer des favoris, lister ses favoris

24. ReviewController : noter / commenter une ressource, lister les avis d'une ressource

25. ConsultationController (ou logique automatique) : enregistrer une consultation à chaque ouverture de ressource

26. Route historique : lister les consultations de l'utilisateur connecté

Test de fin de phase : ajouter aux favoris, noter une ressource, vérifier que l'historique se remplit.

Phase 7 — Recherche et filtres

27. Recherche dans ResourceController@index : par titre, auteur, mots-clés (LIKE ou recherche full-text MySQL)

28. Filtres : par catégorie, type, langue, date

29. Pagination des résultats

```
->paginate(20)
```

30. Tri : par date, popularité (nombre de consultations), note moyenne

Test de fin de phase : rechercher, filtrer et trier via des paramètres d'URL dans Postman.

Phase 8 — Module IA (le plus complexe)

31. Créer le microservice Python/FastAPI séparément :
• Endpoint /analyze : extraction de texte + résumé + suggestion de catégorie/tags
• Endpoint /ask : question-réponse RAG
• Intégration Gemini API + Chroma

32. Configurer les Queues Laravel

```
php artisan queue:table
php artisan migrate
php artisan queue:work
```

33. Créer le Job IndexResourceForAI : appelle le microservice à chaque upload, stocke le résumé et les chunks

```
php artisan make:job IndexResourceForAI
```

34. ConversationController : poser une question sur un document, récupérer l'historique de conversation

35. Gestion des erreurs : timeouts, quota Gemini dépassé (429), microservice indisponible

Test de fin de phase : uploader un document > vérifier que le résumé est généré en arrière-plan > poser une question dessus et recevoir une réponse avec sources citées.

Phase 9 — Statistiques et administration

36. AdminController : gestion des utilisateurs, changement de rôles

37. StatisticsController : ressources les plus consultées, tendances par catégorie, activité globale

38. Dashboard data : les endpoints qui alimenteront le tableau de bord React

Phase 10 — Finalisation

39. Tests : au minimum quelques tests fonctionnels sur les routes critiques (auth, upload, modération)

40. Documentation API : lister toutes les routes (ou générer avec un outil comme Scribe) — utile pour le rapport ET pour développer le frontend

41. Nettoyage : vérifier les autorisations sur toutes les routes, supprimer le code de debug

42. Seeders complets : jeu de données de démonstration pour la soutenance

Conseil sur l'ordre de travail

Les **phases 1 à 5** constituent le cœur du backend. Une fois terminées, le backend est fonctionnel et le développement de React peut démarrer **en parallèle** des phases 6 et suivantes, plutôt que d'attendre 100 % du backend avant de voir quelque chose à l'écran.

La **phase 8 (IA)** est la plus longue et la plus risquée : il vaut mieux commencer à expérimenter le microservice Python dès la fin de la phase 4 (upload), en parallèle du reste, plutôt que de la garder pour la toute fin.